

Contents

Project Title.....	1
1. Project Overview.....	1
2. Key Questions.....	1
3. Datasets.....	1
4. Modelling Workflow.....	2
4.1 Raw data presentation.....	3
4.2 Building and property integration.....	3
4.3 Flood exposure modelling.....	4
4.4 Final modelling population.....	4
4.5 Vulnerability and loss modelling.....	5
5. Results.....	6
5.1 Overview.....	6
5.2 Distribution of property-level losses.....	9
5.3 Depth-damage relationship.....	10
5.4 Illustrative loss-exceedance analysis.....	11
5.5 Spatial distribution of damage.....	12
6. Limitations and Model Development Opportunities.....	14
7. Conclusion.....	14
Appendix.....	16

List of Figures

Figure 1- Number of exposed properties under each flood scenario.....	6
Figure 2- Total estimated flood damage under each flood scenario.....	7
Figure 3- Mean estimated flood damage under each flood scenario.....	8
Figure 4- Distribution of estimated damage across properties.....	9
Figure 5- Flood depth and estimated damage at property level.....	10
Figure 6- Illustrative loss-exceedance probability curve.....	11
Figure 7- Flood depth and estimated damage per borough (1-in-30 River/Sea).....	12
Figure 8- Flood depth and estimated damage per borough (1-in-100 River/1-in200 Sea).....	13

Building-Level Flood Risk and Financial Loss Modelling for London

1. Project Overview

This project tackles UK natural hazard modelling at a granular scale, looking at building-level flood risk and financial loss modelling in London. The project aims to demonstrate a number of tasks and technical skills relevant to catastrophe risk modelling and exposure management. The workflow incorporates the hazard, exposure, vulnerability and financial loss estimation components. However, the project represents a simplified deterministic flood-risk modelling framework rather than a full probabilistic catastrophe model. The core database for the model was constructed in SQL, which was linked to both R and QGIS to create various figures and maps, including an illustrative exceedance probability (EP) curve and associated average annual loss (AAL) calculation. The direction of the project was guided by the four key questions below.

2. Key Questions

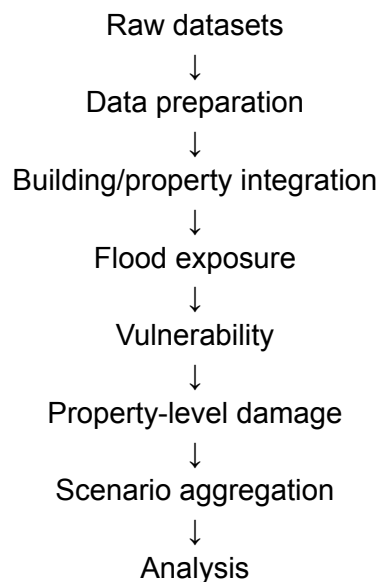
- Q1- Which properties and associated buildings within the study area are exposed to river and sea flooding under the selected defended flood scenarios?
- Q2- How does flood depth relate to estimated property-level damage under different flood scenarios?
- Q3- What are the estimated direct financial losses to exposed properties under different flood scenarios?
- Q4- Which areas contain the greatest concentrations of estimated flood damage under different flood scenarios?

3. Datasets

- **Ordnance Survey, OS OpenMap** - Building polygons were used as the spatial footprint of buildings within the study area, thus providing the basis for identifying buildings intersecting the Environment Agency flood-depth datasets.
- **Ordnance Survey, OS Open UPRN** - was used to provide a unique identifier and spatial location for individual addressable properties. Unique Property Reference Numbers (UPRNs) acted as the primary link between building footprints, Energy Performance Certificate (EPC) records and the subsequent property-level flood estimates.

- **Ministry of Housing, Communities and Local Government (MHCLG), Energy Performance Certificate (EPC)** - domestic EPC records from England and Wales were selected and provided property-level characteristics for residential properties. The latest available UPRN certificate for each was retained, therefore providing information such as: property type, build form, construction age band and total floor area.
- **Environment Agency, Rivers and Sea defended and undefended flood risk depths** - Flood exposure represented using depth bands for various return periods. For this project, the 1-in-30 year river/sea as well as the 1-in-100 year river/1-in-200 year sea defended scenarios were chosen from the broader dataset.
- **Ordnance Survey, OS London Boroughs boundary data** - these boundaries defined the study area and properties were assigned to boroughs for spatial aggregation and visualisation.
- **Middlesex University Flood Hazard Research Centre, Multi-Coloured Manual (MCM) depth-damage data** - residential flood-depth damage relationships were obtained from the MCM.
- **Office for National Statistics (ONS), Consumer Price Index (CPI)** - used to update damage estimates from 2013 prices to August 2026 prices.

4. Modelling Workflow



Tool	Application
PostgreSQL	Data integration, spatial joins, exposure modelling and loss calculation
QGIS	Spatial inspection and cartographic outputs
R	Data analysis, summarisation and visualisation

4.1 Raw data presentation

The raw datasets were reviewed and the relevant fields identified; these chosen fields were then progressed for subsequent exposure and loss modelling. The original raw datasets were retained in a separate section of the SQL database. This preserved original source information, for instance the Environment Agency's 'unavailable' flood depth classification or missing values within the EPC dataset.

Data-types were converted where necessary to ensure reliable joins between datasets. One example was the UPRNs contained within the EPC dataset being converted from 'TEXT' to 'BIGINT' which meant they could be consistently matched with the numeric UPRNs within the OS open UPRN dataset. This was especially important because the UPRN was used as the primary property-level identifier which linked the address, building and EPC datasets.

PostGIS (a PostgreSQL spatial extension package) was used to prepare spatial datasets using a common British National Grid coordinate reference system (EPSG:27700). UPRN coordinate pairs were converted into PostGIS points, this allowed them to become spatially related to OS building footprints as well as other polygon datasets. Additionally, spatial indexes were created for the geometry columns used in the spatial joins, this helped ensure sound query performance.

4.2 Building and property integration

An important distinction made in the exposure model is between physical building footprints and individual properties. OS Open Map Local provided the building footprint geometry, while OS Open UPRN provided property-level identifiers. Multiple UPRNs can occur within a single building (i.e. a block of flats), therefore the two datasets were linked spatially rather than assuming a one-to-one relationship. Within PostGIS, UPRN point geometries were spatially joined to building polygons. If the coordinate of a UPRN lay within a corresponding building footprint, the UPRN was assigned to that building, eventually producing a building-UPRN relationship table. This spatial join resulted in ~7.2 million building-UPRN relationships across ~1.3 million buildings.

The resulting UPRN identifiers were then linked to the domestic EPC dataset. This meant that properties were given associated EPC characteristics such as: property type, build form and

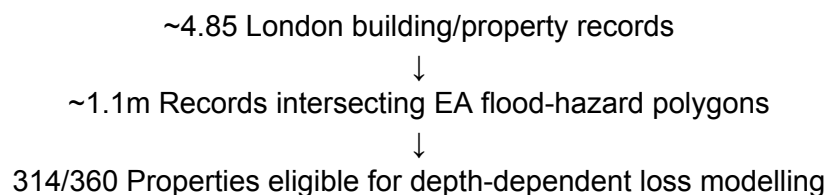
construction age band. In cases where multiple EPC certificates existed for the same UPRN, the most recently lodged certificate was retained. Finally, these building-property records were spatially intersected with the OS London borough boundary dataset, defining the study area and assigning each a corresponding borough. The “ST_Intersects” function was used here, which meant that buildings which lay on the borough boundaries weren’t excluded from the study population.

4.3 Flood exposure modelling

Two present-day Environment Agency flood scenarios were modelled with differing return periods: 1-in-30 Rivers/Sea and 1-in-100 river/1-in-200 sea. These were both ‘defended’ scenarios, which assume that implemented flood defences work as intended. Both undefended and surface-water scenarios were outside the scope of the analysis. A spatial operation was then conducted, flood hazard polygons (one for each scenario) were spatially intersected with the building geometries. Properties which intersected a flood hazard polygon were therefore considered exposed under the corresponding scenario. A single building or property could intersect multiple Environment Agency hazard polygons, potentially producing several records with different depth bands. Treating each intersection as a separate loss would risk double-counting the same property. To resolve this, the maximum applicable flood-depth band was retained for each UPRN within each scenario, producing one flood-depth classification for each property.

4.4 Final modelling population

The application of the spatial, exposure and data-availability criteria resulted in a substantially reduced final modelling population compared with the initial datasets. Firstly, only properties/buildings within the London study area which could be linked to individual UPRNs were usable. Secondly, only properties with building footprints which intersected the defended flood hazard dataset polygons would be eligible. It is important to note that the Environment Agency datasets contain extensive areas which are classified as “unavailable” (i.e. no usable flood-depth value is provided) therefore eliminating an extensive number of properties from the final modelling population as the depth-damage calculation could not be performed. Following these stages, the final modelling population consisted of 314 properties under the 1-in-30 defended Rivers/Sea scenario and 360 properties under the 1-in-100 river / 1-in-200 sea defended scenario. The process is summarised below...



4.5 Vulnerability and Loss Modelling

In this context vulnerability is the susceptibility of an exposed property to damage when subjected to a given flood hazard. For this project, vulnerability was represented using residential depth-damage relationships from the Multi-Coloured Manual (MCM). MCM damage values were used to assign an estimated monetary damage to each exposed property based on the depth band it was exposed to.

There was not perfect correspondence between the Environment Agency return-period scenarios chosen and the available return-period tables in the MCM. As a result, the 1-in-30 year EA scenario was mapped to the MCM 25 year return period table and the 1-in-100 river/ 1-in-200 sea EA scenario used the MCM 100 year return period table. The latter represents a proxy based primarily on the river component of the combined scenario and is therefore an explicit modelling approximation. It was also necessary to harmonise the depth bands of the EA data and MCM depth-damage tables, for instance the EA 1,200–2,300 mm and >2,300 mm categories were both mapped to the MCM >1,200 mm category because the MCM data did not provide further depth differentiation at these higher depths.

The MCM depth-damage figures were expressed in 2013 prices, therefore the values were adjusted using the ONS Consumer Price Index to August 2026 prices. This meant that each property had an estimate of flood damage under each modelled flood scenario. It is important to clarify that these are direct economic damages rather than actual predicted insurance claims.

The resulting property-level loss tables provided the basis for subsequent loss aggregation, scenario comparison and spatial analysis. R was subsequently used for loss aggregation, scenario comparison and visualisation, while QGIS was used to produce the final spatial risk maps.

5. Results

5.1 Overview

Figure 1 shows the number of properties identified as exposed under each defended flood scenario. The 1-in-30 scenario produces 314 exposed properties, compared with 360 under the 1-in-100 river / 1-in-200 sea scenario. The less frequent scenario therefore identifies 46 additional properties, representing an increase of approximately 14.6% in the final loss-modelling population.

Figure 1 - Number of Exposed Properties under each Flood Scenario

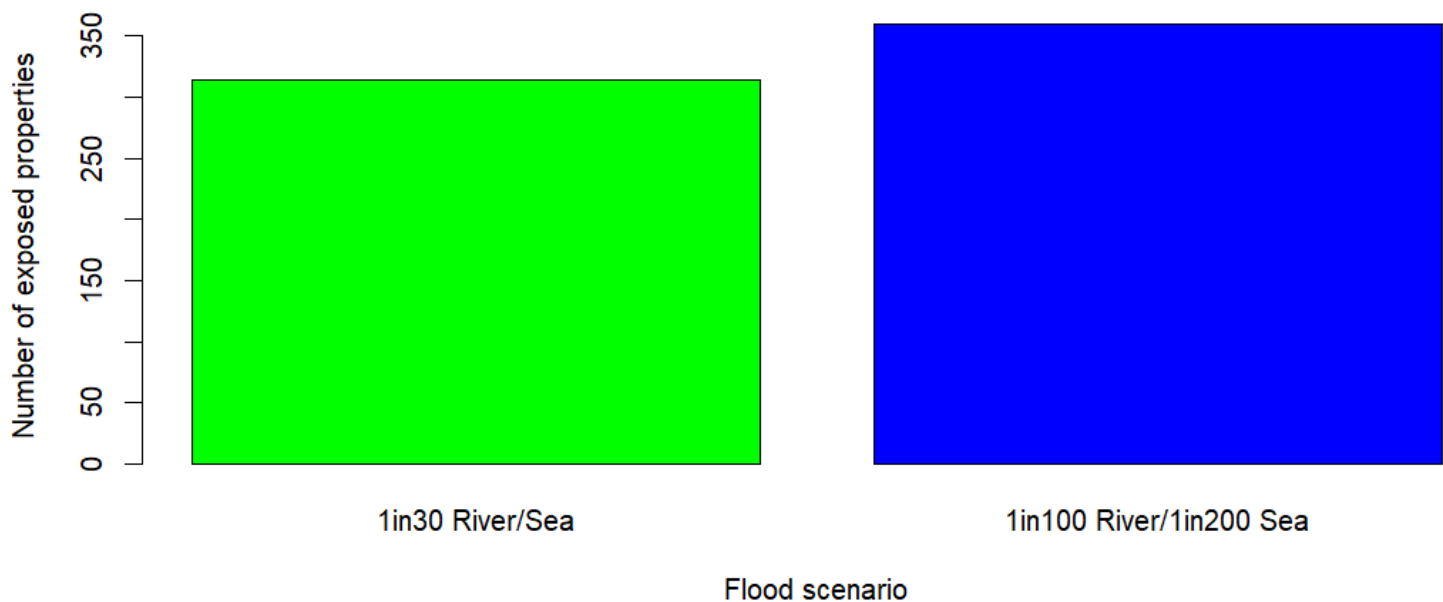


Figure 2 shows the corresponding estimated direct flood damage under each scenario. Total estimated damage increases from approximately £10.94 million under the 1-in-30 scenario to £13.74 million under the 1-in-100/200 scenario, an increase of approximately 25.5%.

Figure 2 - Total Estimated Damage under each Flood Scenario

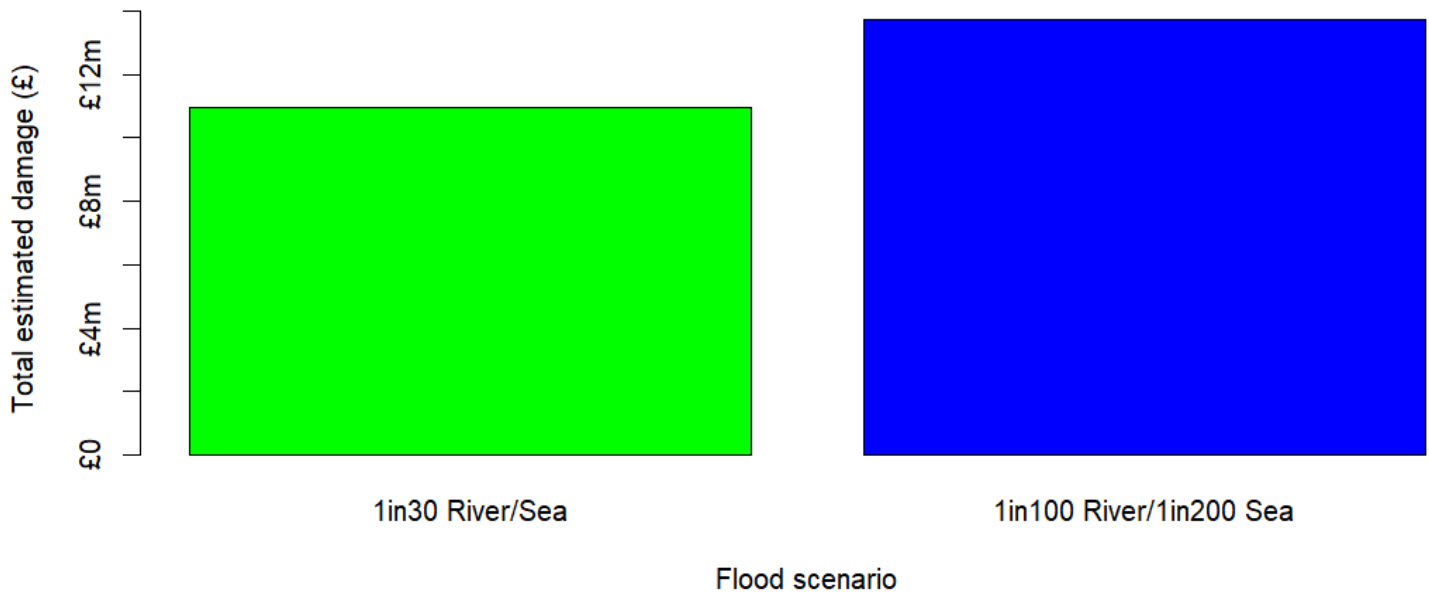
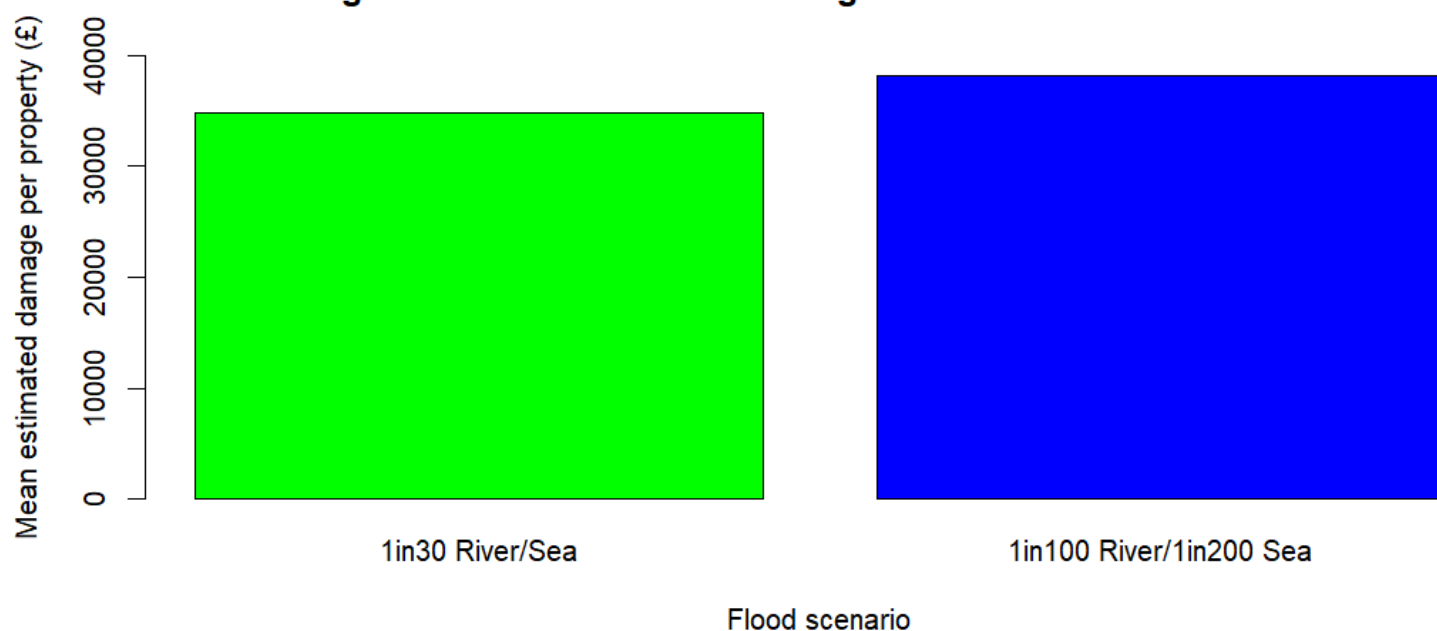


Figure 3 shows mean estimated damage per exposed property. Mean loss increases from approximately £34,850 to £38,156 between the two scenarios, indicating that the increase in aggregate damage reflects both a larger exposed population and higher average loss among exposed properties.

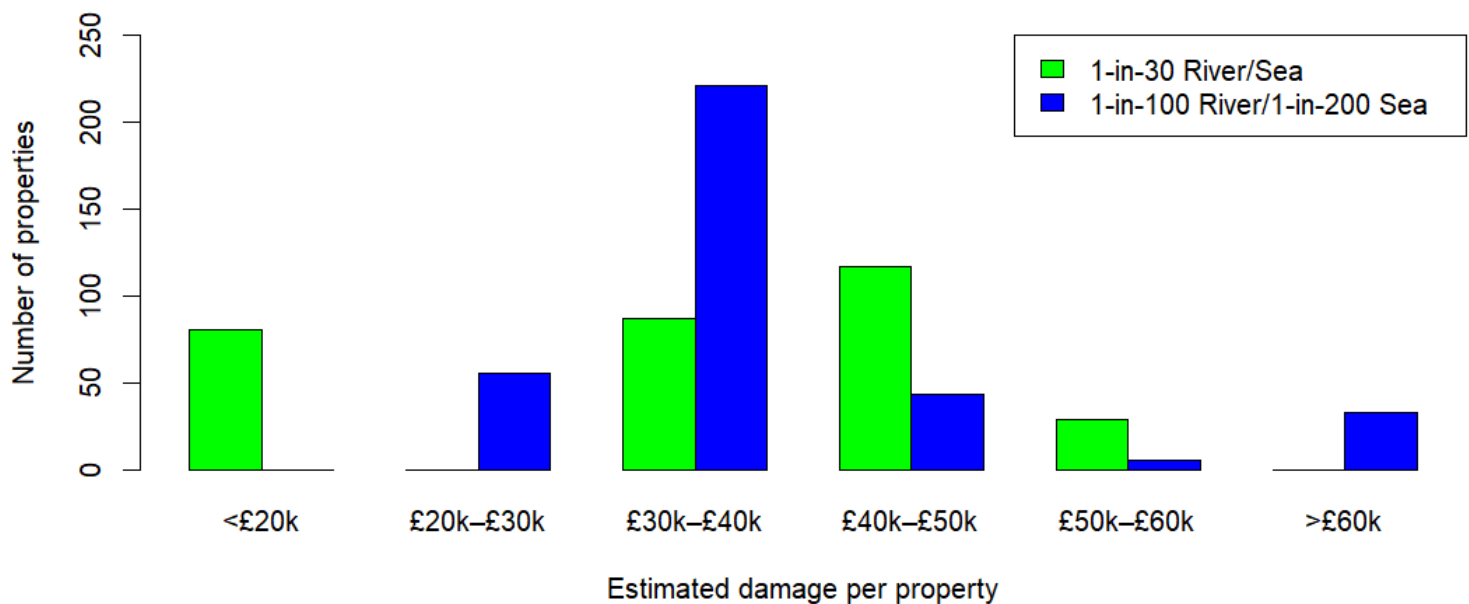
Figure 3 - Mean Estimated Damage under each Flood Scenario



5.2 Distribution of property-level losses

Figure 4 shows the distribution of estimated damage across individual properties under the two flood scenarios. The 1-in-30 scenario is concentrated primarily between £30,000 and £50,000 per property, whereas the 1-in-100 river / 1-in-200 sea scenario produces a wider distribution, including properties with estimated losses exceeding £60,000. This indicates that the higher aggregate damage under the broader scenario reflects not only a greater number of exposed properties, but also a shift towards higher individual loss severities.

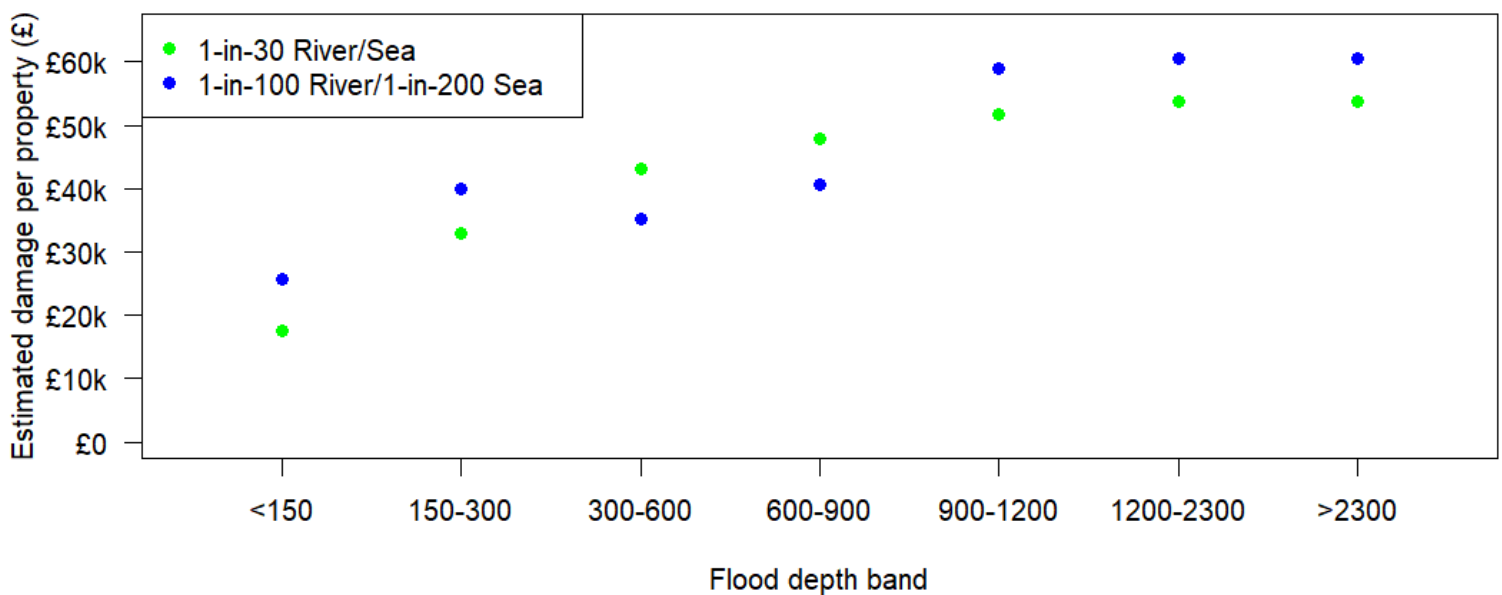
Figure 4 - Distribution of Estimated Damage across Properties



5.3 Depth-Damage relationship

Figure 5 illustrates the relationship between flood depth and estimated property-level damage. Under the 1-in-30 scenario, estimated damage generally increases with flood depth, reflecting the underlying depth-damage relationship used in the vulnerability model. The 1-in-100 river / 1-in-200 sea scenario follows the same general pattern, although some depth bands show variation in estimated damage due to the source vulnerability values and the different distribution of properties across depth categories.

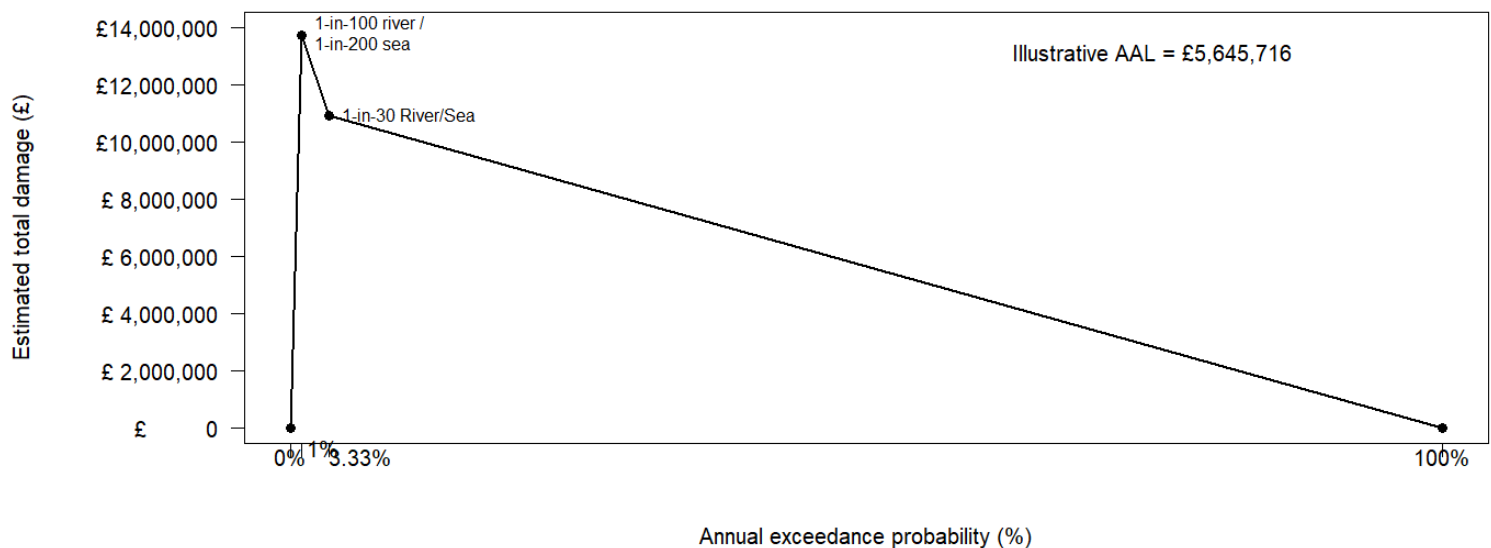
Figure 5 - Flood Depth and Estimated Damage at Property Level



5.4 Illustrative loss-exceedance analysis

Figure 6 presents an illustrative loss–exceedance probability curve constructed from the modelled flood scenarios. The curve demonstrates the relationship between event probability and aggregate estimated damage, with the less frequent 1-in-100/200 scenario producing greater total loss than the 1-in-30 scenario. An illustrative average annual loss (AAL) of approximately £5.65 million is obtained from the assumed interpolation between the available scenarios. This value should be interpreted as indicative rather than as a full catastrophe-model AAL, given the limited number of modelled return periods and the assumptions required to construct the curve.

Figure 6- Illustrative Loss–Exceedance Probability Curve



5.5 Spatial distribution of damage

Figure 7 shows the spatial distribution of estimated damage under the 1-in-30 defended Rivers/Sea scenario. Estimated losses are concentrated within a relatively small number of London boroughs, with Croydon containing the largest concentration of modelled damage. The map also illustrates the spatial relationship between estimated damage and the underlying flood-depth hazard, providing a geographic view of where the modelled financial impacts are concentrated.



Figure 7 – Flood Depth and Estimated Damage by Borough (1-in-30 Defended River/Sea)

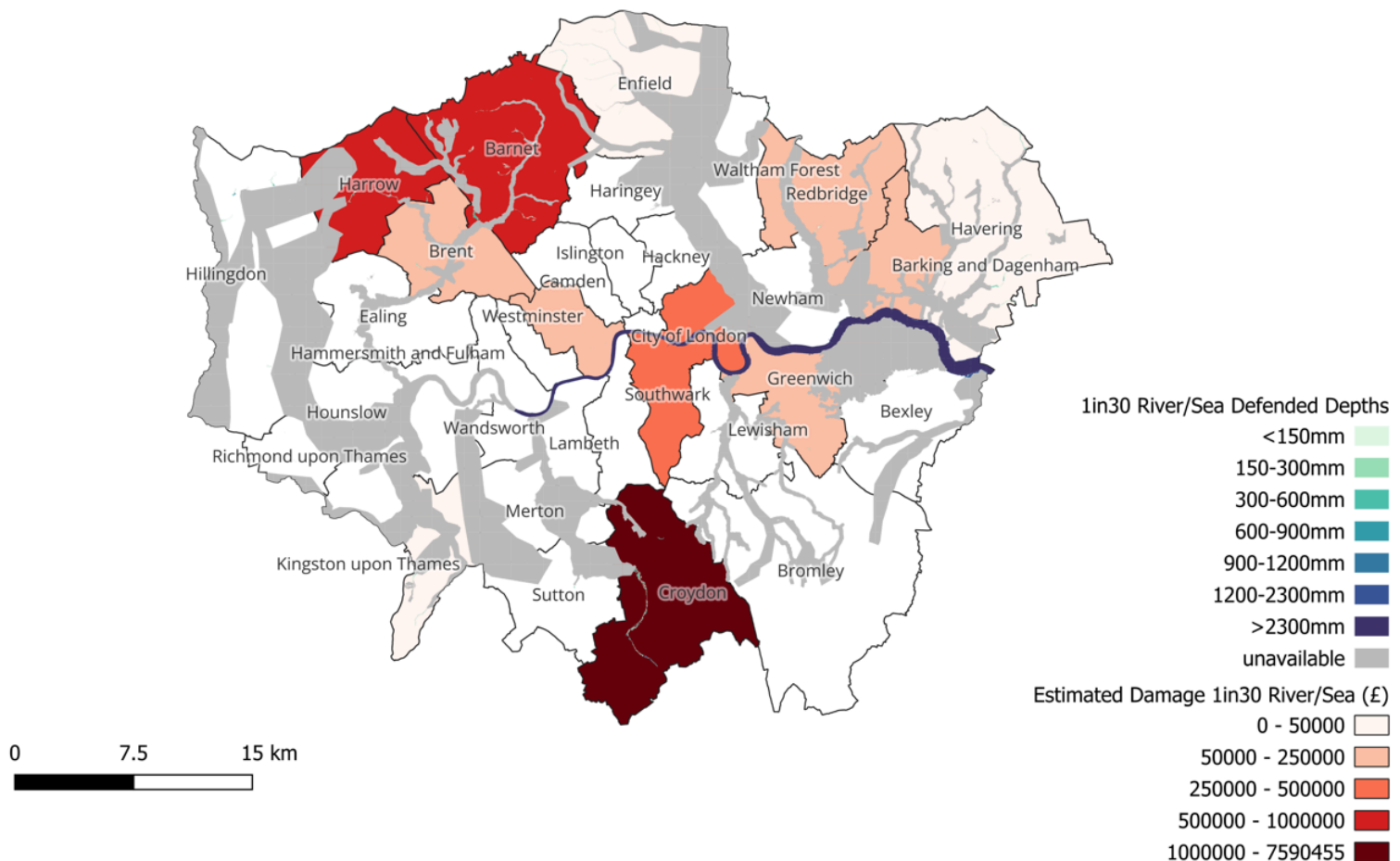
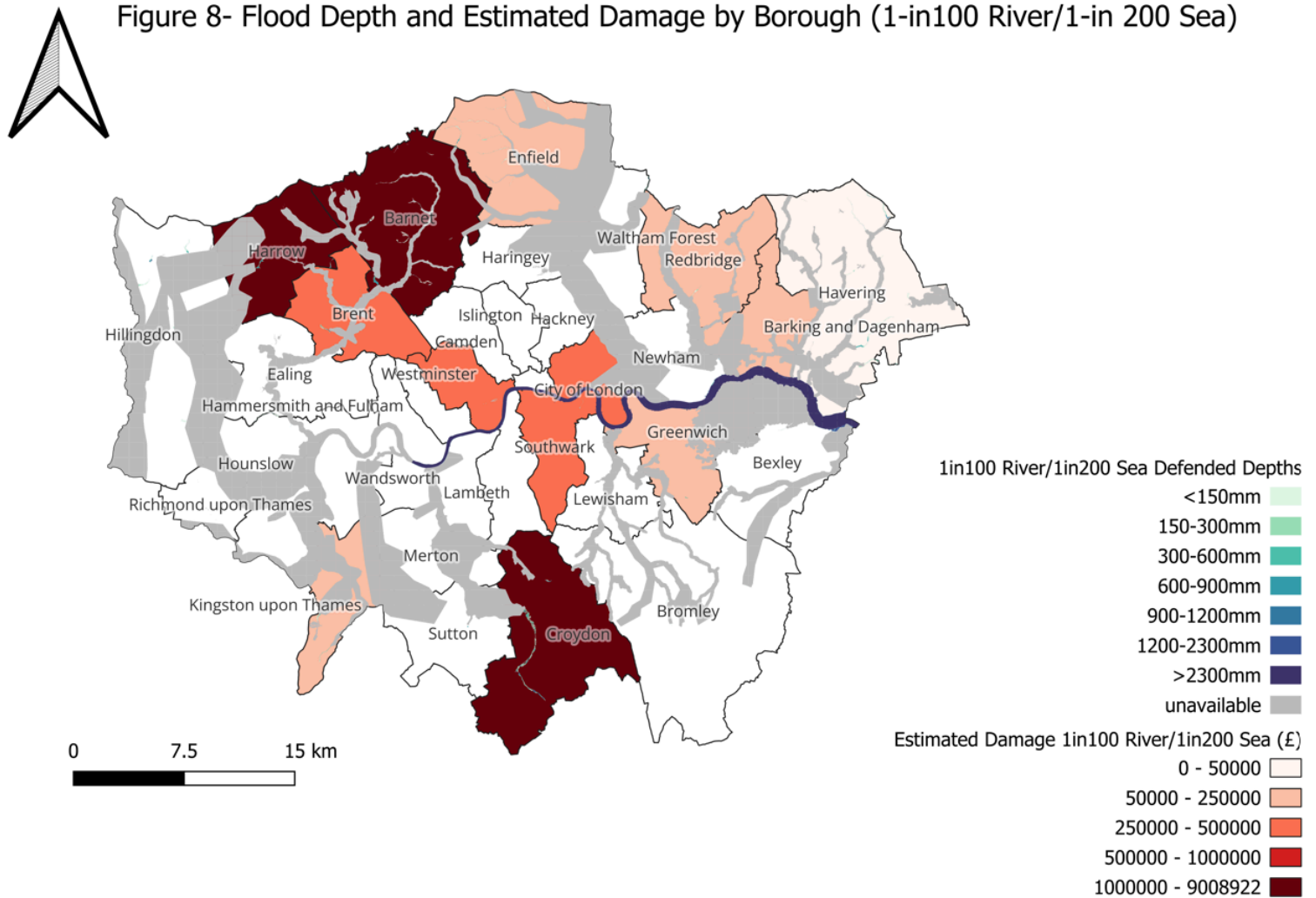


Figure 8 shows the corresponding spatial distribution under the 1-in-100 river / 1-in-200 sea defended scenario. The spatial pattern remains concentrated in several boroughs, but the less frequent scenario produces additional exposed properties and increased estimated damage across the affected area. Comparing Figures 7 and 8 therefore demonstrates how the spatial distribution of modelled financial loss changes between the two hazard scenarios.



6. Limitations and Model Development Opportunities

One of the largest limitations of the study was the high proportion of “unavailable” entries within the Environment Agency flood-depth data. These records could be identified as intersecting the flood-hazard datasets but could not be progressed to depth-dependent damage estimation. This, combined with the requirement for the property-level information necessary to apply the vulnerability model, resulted in a substantially smaller final modelling population than the initial London-wide building/property dataset. This had an especially large impact on the results shown in figures 7 and 8. This is because some boroughs that are clearly exposed to large amounts of flooding under both scenarios show a damage score of £0 because no depth data were available to produce damage calculations.

There were also a number of methodological limitations. Firstly, the vulnerability component of flood risk is more multi-faceted than flood depth alone. Additional property characteristics available within the EPC dataset could potentially have been incorporated as vulnerability modifiers, but doing so would have required a more complex vulnerability methodology than the depth-damage approach adopted here. There were also substantial assumptions involved in standardising the return periods and depth bands between the EA datasets and the MCM data tables, which could have affected the accuracy of the results.

To keep the scope of the project manageable, only two hazard/loss scenarios were modelled, although in an industry standard catastrophe model far more scenarios would be used. This had an impact on the robustness of the exceedance probability curve and associated average annual loss calculation. Another modelling decision taken in the interests of keeping the project's scope manageable was to model only domestic properties, excluding commercial and public buildings. Including these property types would produce a more representative exposure portfolio and provide a broader assessment of flood risk across the study area.

To summarise, the inclusion of the following considerations could develop the model substantially: additional return periods, stochastic event sets, climate change scenarios, more diverse building vulnerability considerations, public and commercial exposure (instead of only domestic) and probabilistic event modelling and loss distributions.

7. Conclusion

This project developed a reproducible building/property-level flood risk modelling workflow for London using PostgreSQL/PostGIS, QGIS and R. By integrating granular exposure data with Environment Agency flood-depth scenarios and MCM depth-damage relationships, the model produced scenario-specific estimates of direct flood damage and spatial concentrations of loss. This project undertakes many aspects of a typical industry workflow (i.e. Hazard → Exposure → Vulnerability → Financial Loss → Spatial Aggregation), demonstrating experience with: granular exposure construction, spatial hazard overlays, property-level identifiers, vulnerability

relationships, scenario-based loss estimation, database-driven spatial modelling, loss aggregation and loss-exceedance analysis.

The model is deliberately a scenario-based deterministic framework rather than a full commercial catastrophe model. It uses a limited number of flood scenarios and a simplified vulnerability representation, meaning the resulting losses should be interpreted as modelled estimates rather than predictions of actual insurance claims. Nevertheless, the workflow showcases several core CAT modelling principles and provides a foundation that could be extended through additional return periods, more detailed vulnerability functions, richer exposure attributes, uncertainty analysis and probabilistic event modelling.

Appendix

```
-- Importing data
CREATE EXTENSION postgis;
SELECT PostGIS_Version();
CREATE SCHEMA raw;
CREATE SCHEMA model;
CREATE SCHEMA results;
SELECT postGIS_Version();

SELECT * FROM raw.os_buildings LIMIT 10;
SELECT COUNT (*) FROM raw.os_buildings
SELECT id, featcode FROM raw.os_buildings LIMIT 10;
SELECT ST_SRID(geom) FROM raw.os_buildings LIMIT 1;

SELECT * FROM raw.flood_hazard_1in30 LIMIT 10;
SELECT * FROM raw.london_boroughs LIMIT 10;

CREATE TABLE raw.os_uprn (uprn BIGINT,x_coordinate DOUBLE PRECISION, y_coordinate
DOUBLE PRECISION,latitude DOUBLE PRECISION,longitude DOUBLE PRECISION);
COPY raw.os_uprn (uprn,x_coordinate,y_coordinate,latitude,longitude) FROM
'C:\Users\tobyj\OneDrive\Documents\Project 2\uprn\osopenuprn_202608.csv' WITH (FORMAT
csv,HEADER true);
SELECT * FROM raw.os_uprn LIMIT 10;

CREATE TABLE raw.epc_domestic (certificate_number TEXT, address1 TEXT, address2 TEXT,
address3 TEXT, postcode TEXT, posttown TEXT, address TEXT, constituency TEXT,
constituency_label TEXT, local_authority TEXT, local_authority_label TEXT, built_form TEXT,
co2_emiss_curr_per_floor_area TEXT, co2_emissions_current TEXT, co2_emissions_potential
TEXT, construction_age_band TEXT, current_energy_efficiency TEXT, current_energy_rating
TEXT, energy_consumption_current TEXT, energy_consumption_potential TEXT, energy_tariff
TEXT, environment_impact_current TEXT, environment_impact_potential TEXT,
extension_count TEXT, fixed_lighting_outlets_count TEXT, flat_storey_count TEXT,
flat_top_storey TEXT, floor_description TEXT, floor_energy_eff TEXT, floor_height TEXT,
floor_level TEXT, glazed_area TEXT, glazed_type TEXT, heat_loss_corridor TEXT,
heating_cost_current TEXT, heating_cost_potential TEXT, hot_water_cost_current TEXT,
hot_water_cost_potential TEXT, hot_water_energy_eff TEXT, hot_water_env_eff TEXT,
hotwater_description TEXT, inspection_date TEXT, lighting_cost_current TEXT,
lighting_cost_potential TEXT, lighting_description TEXT, lighting_energy_eff TEXT,
lighting_env_eff TEXT, lodgement_date TEXT, lodgement_datetime TEXT, low_energy_lighting
TEXT, low_energy_fixed_lighting_outlets_count TEXT, main_fuel TEXT, mainheat_description
TEXT, mainheat_energy_eff TEXT, mainheat_env_eff TEXT, mainheatc_energy_eff TEXT,
mainheatc_env_eff TEXT, mainheatcont_description TEXT, main_heating_controls TEXT,
mains_gas_flag TEXT, mechanical_ventilation TEXT, multi_glaze_proportion TEXT,
```

```

number_habitable_rooms TEXT, number_heated_rooms TEXT, number_open_fireplaces TEXT,
photo_supply TEXT, potential_energy_efficiency TEXT, potential_energy_rating TEXT,
property_type TEXT, report_type TEXT, roof_description TEXT, roof_energy_eff TEXT,
roof_env_eff TEXT, secondheat_description TEXT, sheating_energy_eff TEXT,
sheating_env_eff TEXT, solar_water_heating_flag TEXT, tenure TEXT, total_floor_area TEXT,
transaction_type TEXT, unheated_corridor_length TEXT, walls_description TEXT,
walls_energy_eff TEXT, walls_env_eff TEXT, wind_turbine_count TEXT, windows_description
TEXT, windows_energy_eff TEXT, windows_env_eff TEXT, floor_env_eff TEXT, region TEXT,
country TEXT, uprn TEXT, uprn_source TEXT);SELECT COUNT(*) FROM raw.epc_domestic;
SELECT uprn, property_type, built_form, total_floor_area, postcode, construction_age_band
FROM raw.epc_domestic LIMIT 10;

```

```

CREATE TABLE raw.ons_london_property_values (region_country_code
TEXT,region_country_name TEXT,local_authority_code TEXT,local_authority_name
TEXT,property_type TEXT,period TEXT,median_price NUMERIC);
SELECT * FROM raw.ons_london_property_values LIMIT 10;

```

```

SELECT * FROM raw.flood_hazard_rivers_1in100_sea_1in200_defended LIMIT 10;

```

```

-- Validate database

```

```

-- Check all data tables are present

```

```

SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'raw'
ORDER BY table_name;

```

```

--Check how much data is in each table

```

```

SELECT 'epc_domestic' AS table_name, COUNT(*) AS rows FROM raw.epc_domestic
UNION ALL
SELECT 'flood_hazard_1in30_defended', COUNT(*) FROM raw.flood_hazard_1in30_defended
UNION ALL
SELECT 'flood_hazard_rivers_1in100_sea_1in200_defended', COUNT(*) FROM
raw.flood_hazard_rivers_1in100_sea_1in200_defended
UNION ALL
SELECT 'london_boroughs', COUNT(*) FROM raw.london_boroughs
UNION ALL
SELECT 'ons_london_property_values', COUNT(*) FROM raw.ons_london_property_values
UNION ALL
SELECT 'os_buildings', COUNT(*) FROM raw.os_buildings
UNION ALL
SELECT 'os_uprn', COUNT(*) FROM raw.os_uprn;

```

```

-- Check columns were interpreted correctly

```

```
SELECT table_name, column_name, data_type FROM information_schema.columns WHERE
table_schema = 'raw' ORDER BY table_name, ordinal_position;
```

```
-- Check spatial data
```

```
SELECT f_table_schema, f_table_name, f_geometry_column, srid, type FROM
geometry_columns WHERE f_table_schema = 'raw' ORDER BY f_table_name;
```

```
SELECT
  'os_buildings' AS table_name,
  COUNT(*) AS invalid_geometries
```

```
-- Check for a link
```

```
SELECT COUNT(*) AS total_rows, COUNT(DISTINCT uprn) AS unique_uprns FROM
raw.os_uprn;
```

```
-- Check for missing values
```

```
SELECT COUNT(*) AS missing_uprns FROM raw.os_uprn WHERE uprn IS NULL;
```

```
--
```

```
SELECT COUNT(*) AS total_epc, COUNT(uprn) AS epc_with_uprn, COUNT(*) - COUNT(uprn)
AS epc_without_uprn FROM raw.epc_domestic;
```

```
-- Check for duplicates
```

```
SELECT COUNT(*) AS uprns_with_multiple_certificates FROM (SELECT uprn FROM
raw.epc_domestic WHERE uprn IS NOT NULL GROUP BY uprn HAVING COUNT(*) > 1) AS
duplicates;
```

```
-- Check for distinct
```

```
SELECT COUNT(DISTINCT uprn) AS unique_epc_uprns FROM raw.epc_domestic WHERE
uprn IS NOT NULL;
```

```
-- Check for links
```

```
SELECT COUNT(DISTINCT e.uprn) AS epc_uprns_found_in_os FROM raw.epc_domestic e
INNER JOIN raw.os_uprn u ON e.uprn::BIGINT = u.uprn WHERE e.uprn IS NOT NULL;
```

```
--
```

```
SELECT COUNT(DISTINCT local_authority_code) AS unique_london_authorities,
COUNT(DISTINCT local_authority_name) AS unique_authority_names FROM
raw.ons_london_property_values;
```

```
-- Check for missing data
```

```
SELECT COUNT(*) AS total_values, COUNT(median_price) AS non_null_values, COUNT(*) -
COUNT(median_price) AS missing_values, COUNT(*) FILTER (WHERE median_price <= 0)
AS zero_or_negative_values FROM raw.ons_london_property_values;
```

```

-- Check where missing data is concentrated
SELECT property_type, COUNT(*) AS missing_values FROM raw.ons_london_property_values
WHERE median_price IS NULL GROUP BY property_type ORDER BY missing_values DESC;

--
SELECT depth_band, COUNT(*) AS records FROM raw.flood_hazard_1in30_defended
GROUP BY depth_band ORDER BY depth_band;
SELECT depth_band, COUNT(*) AS records FROM
raw.flood_hazard_rivers_1in100_sea_1in200_defended GROUP BY depth_band ORDER BY
depth_band;
SELECT flood_sour, COUNT(*) AS records FROM raw.flood_hazard_1in30_defended GROUP
BY flood_sour ORDER BY flood_sour;
SELECT flood_sour, COUNT(*) AS records FROM
raw.flood_hazard_rivers_1in100_sea_1in200_defended GROUP BY flood_sour ORDER BY
flood_sour;

--END OF DATA CLEANING/VALIDATION

-- Create first model dataset from the original raw one
CREATE TABLE model.buildings AS SELECT id AS building_id, geom, featcode FROM
raw.os_buildings;
-- Create spatial index for the table, so we can do spatial operations
CREATE INDEX buildings_geom_idx ON model.buildings USING GIST (geom);

--
CREATE TABLE model.uprn AS SELECT uprn, x_coordinate, y_coordinate, latitude, longitude,
ST_SetSRID (ST_MakePoint(x_coordinate, y_coordinate), 27700) AS geom FROM
raw.os_uprn;

--Create an index
CREATE INDEX uprn_geom_idx ON model.uprn USING GIST (geom);

-- Create the model table for epc
CREATE TABLE model.epc AS SELECT certificate_number, uprn::BIGINT AS uprn,
property_type, built_form, construction_age_band, total_floor_area::NUMERIC AS
total_floor_area, postcode, local_authority, current_energy_rating FROM raw.epc_domestic
WHERE uprn IS NOT NULL;

-- Create the model table for property values
CREATE TABLE model.property_values AS SELECT local_authority_code,
local_authority_name, property_type, period, median_price FROM
raw.ons_london_property_values;

```

```

-- Create the model table for london boroughs
CREATE TABLE model.boroughs AS SELECT id, name, gss_code, geom FROM
raw.london_boroughs;

-- Create the model tables for the low and high retrain period floods
CREATE TABLE model.flood_hazard_1in30_defended AS SELECT id, geom, depth_band,
flood_sour FROM raw.flood_hazard_1in30_defended;
CREATE TABLE model.flood_hazard_rivers_1in100_sea_1in200_defended AS SELECT id,
geom, depth_band, flood_sour FROM raw.flood_hazard_rivers_1in100_sea_1in200_defended;

-- Create the spatial index's for the two new tables
CREATE INDEX flood_1in30_geom_idx ON model.flood_hazard_1in30_defended USING GIST
(geom);
CREATE INDEX flood_1in100_200_geom_idx ON
model.flood_hazard_rivers_1in100_sea_1in200_defended USING GIST (geom);

-- Test before beginning joins
SELECT b.building_id, COUNT(u.uprn) AS uprns_inside FROM (SELECT * FROM
model.buildings LIMIT 100) AS b LEFT JOIN model.uprn AS u ON ST_Within(u.geom, b.geom)
GROUP BY b.building_id ORDER BY uprns_inside DESC;

EXPLAIN ANALYZE SELECT b.building_id, COUNT(u.uprn) AS uprns_inside FROM (SELECT
* FROM model.buildings LIMIT 100) AS b LEFT JOIN model.uprn AS u ON u.geom && b.geom
AND ST_Within(u.geom, b.geom) GROUP BY b.building_id ORDER BY uprns_inside DESC;

-- Build the building UPRN linkage table
CREATE TABLE model.building_uprn AS SELECT b.building_id, u.uprn FROM model.buildings
AS b JOIN model.uprn AS u ON u.geom && b.geom AND ST_Within(u.geom, b.geom);

-- Check results from the table
SELECT COUNT(*) AS building_uprn_matches, COUNT(DISTINCT building_id) AS
buildings_with_uprn, COUNT(DISTINCT uprn) AS unique_uprns FROM model.building_uprn;

-- Check before adding the EPC link
SELECT COUNT(*) AS building_uprn_matches, COUNT(e.uprn) AS matches_with_epc,
COUNT(DISTINCT e.uprn) AS unique_epc_uprns FROM model.building_uprn AS bu LEFT
JOIN model.epc AS e ON bu.uprn = e.uprn;

-- Column check
SELECT column_name, data_type FROM information_schema.columns WHERE table_schema
= 'raw' AND table_name = 'epc_domestic' AND (column_name ILIKE '%date%' OR
column_name ILIKE '%lodged%' OR column_name ILIKE '%inspection%' OR column_name
ILIKE '%transaction%') ORDER BY ordinal_position;

```

```

-- Data check
SELECT uprn, certificate_number, inspection_date, lodgement_date, lodgement_datetime,
transaction_type FROM model.epc LIMIT 10;

-- Add columns to table
ALTER TABLE model.epc ADD COLUMN inspection_date TEXT, ADD COLUMN
lodgement_date TEXT, ADD COLUMN lodgement_datetime TEXT;

-- Populate the new columns
UPDATE model.epc AS e SET inspection_date = r.inspection_date, lodgement_date =
r.lodgement_date, lodgement_datetime = r.lodgement_datetime FROM raw.epc_domestic AS r
WHERE e.certificate_number = r.certificate_number;

-- Check if adding the columns was successful
SELECT uprn, certificate_number, inspection_date, lodgement_date, lodgement_datetime
FROM model.epc LIMIT 10;

-- For each UPRN, retain the most recently lodged EPC certificate
SELECT uprn, certificate_number, lodgement_datetime, ROW_NUMBER() OVER (PARTITION
BY uprn ORDER BY lodgement_datetime DESC) AS epc_rank FROM model.epc WHERE uprn
IN ( SELECT uprn FROM model.epc GROUP BY uprn HAVING COUNT(*) > 1) LIMIT 20;

-- Create a temporary table before altering the epc table
CREATE TABLE model.epc_clean AS SELECT certificate_number, uprn, property_type,
built_form, construction_age_band, total_floor_area, postcode, local_authority,
current_energy_rating, inspection_date, lodgement_date, lodgement_datetime FROM (SELECT
*,ROW_NUMBER() OVER (PARTITION BY uprn ORDER BY lodgement_datetime DESC) AS
epc_rank FROM model.epc) AS ranked_epc WHERE epc_rank = 1;

-- Check there is now one epc per UPRN
SELECT COUNT(*) AS total_records, COUNT(DISTINCT uprn) AS unique_uprns FROM
model.epc_clean;

-- Replace the epc table
ALTER TABLE model.epc RENAME TO epc_old;

-- Get rid of old epc table
DROP TABLE model.epc_old;

-- Rename the cleaned table to the new general one
ALTER TABLE model.epc_clean RENAME TO epc;

-- Final verification of the epc data table

```

```
SELECT COUNT(*) AS building_uprn_matches, COUNT(e.uprn) AS matches_with_epc,
COUNT(DISTINCT e.uprn) AS unique_epc_uprns FROM model.building_uprn AS bu LEFT
JOIN model.epc AS e ON bu.uprn = e.uprn;
```

```
ALTER TABLE model.epc ADD COLUMN local_authority_label TEXT;
UPDATE model.epc AS e SET local_authority_label = r.local_authority_label FROM
raw.epc_domestic AS r WHERE e.certificate_number = r.certificate_number;
```

-- Create model.building_properties, this will bring the EPC characteristics onto each building-UPRN relationship.

```
CREATE TABLE model.building_properties AS SELECT bu.building_id, bu.uprn,
e.property_type, e.built_form, e.construction_age_band, e.total_floor_area, e.postcode,
e.local_authority, e.local_authority_label FROM model.building_uprn AS bu LEFT JOIN
model.epc AS e ON bu.uprn = e.uprn;
```

-- Check the building_properties table

```
SELECT COUNT(*) AS total_records, COUNT(DISTINCT building_id) AS unique_buildings,
COUNT(DISTINCT uprn) AS unique_uprns, COUNT(property_type) AS records_with_epc
FROM model.building_properties;
SELECT * FROM model.building_properties LIMIT 10;
SELECT * FROM model.building_properties WHERE property_type IS NOT NULL LIMIT 10;
```

-- Create another table which pins the building_properties table to our boroughs dataset

```
CREATE TABLE model.london_building_properties AS SELECT bp.building_id, bp.uprn,
bp.property_type, bp.built_form, bp.construction_age_band, bp.total_floor_area, bp.postcode,
bp.local_authority, bp.local_authority_label, b.name AS borough_name, b.gss_code AS
borough_gss_code FROM model.building_properties AS bp JOIN model.buildings AS buildings
ON bp.building_id = buildings.building_id JOIN model.boroughs AS b ON
ST_Intersects(buildings.geom, b.geom);
```

-- Check the new table

```
SELECT COUNT(*) AS total_records, COUNT(DISTINCT building_id) AS unique_buildings,
COUNT(DISTINCT uprn) AS unique_uprns, COUNT(property_type) AS records_with_epc
FROM model.london_building_properties;
SELECT borough_name, COUNT(*) AS records FROM model.london_building_properties
GROUP BY borough_name ORDER BY borough_name;
```

-- Drop old table

```
DROP TABLE model.building_properties;
```

-- Create the 1-in-30-year defended flood exposure table, spatially intersect the London building footprints with the EA flood polygons

```
CREATE TABLE model.flood_exposure_1in30 AS SELECT lbp.building_id, lbp.uprn, fh.id AS flood_id, fh.depth_band, fh.flood_sour FROM model.london_building_properties AS lbp JOIN model.buildings AS b ON lbp.building_id = b.building_id JOIN model.flood_hazard_1in30_defended AS fh ON ST_Intersects(b.geom, fh.geom);
```

-- 1-in-30 year exposure table checks

```
SELECT COUNT(*) AS total_records, COUNT(DISTINCT building_id) AS unique_buildings, COUNT(DISTINCT uprn) AS unique_uprns, COUNT(depth_band) AS records_with_depth, COUNT(flood_sour) AS records_with_source FROM model.flood_exposure_1in30;
SELECT depth_band, COUNT(*) AS records FROM model.flood_exposure_1in30 GROUP BY depth_band ORDER BY depth_band;
SELECT flood_sour, depth_band, COUNT(*) AS records FROM model.flood_exposure_1in30 GROUP BY flood_sour, depth_band ORDER BY flood_sour, depth_band;
```

-- Create the 1-in-100/200 year defended flood exposure table, spatially interscet the london building footprints with the EA flood polygons

```
CREATE TABLE model.flood_exposure_1in100_200 AS SELECT lbp.building_id, lbp.uprn, fh.id AS flood_id, fh.depth_band, fh.flood_sour FROM model.london_building_properties AS lbp JOIN model.buildings AS b ON lbp.building_id = b.building_id JOIN model.flood_hazard_rivers_1in100_sea_1in200_defended AS fh ON ST_Intersects(b.geom, fh.geom);
```

-- 1-in-100/200 year exposure table checks

```
SELECT COUNT(*) AS total_records, COUNT(DISTINCT building_id) AS unique_buildings, COUNT(DISTINCT uprn) AS unique_uprns, COUNT(depth_band) AS records_with_depth, COUNT(flood_sour) AS records_with_source FROM model.flood_exposure_1in100_200;
SELECT depth_band, COUNT(*) AS records FROM model.flood_exposure_1in100_200 GROUP BY depth_band ORDER BY depth_band;
```

-- Create a table that joins our flood exposure records back to the EPC-linked London properties, while only retaining records with a usable depth band. Thus establishing the actual modelling popultion for each scenario

```
CREATE TABLE model.loss_population_1in30 AS SELECT fe.building_id, fe.uprn, fe.flood_id, fe.depth_band, fe.flood_sour, lbp.property_type, lbp.built_form, lbp.construction_age_band, lbp.total_floor_area, lbp.postcode, lbp.borough_name, lbp.borough_gss_code FROM model.flood_exposure_1in30 AS fe JOIN model.london_building_properties AS lbp ON fe.building_id = lbp.building_id AND fe.uprn = lbp.uprn WHERE fe.depth_band <> 'unavailable' AND lbp.property_type IS NOT NULL;
```

-- How many properties made it through the filter

```
SELECT COUNT(*) AS total_records, COUNT(DISTINCT building_id) AS unique_buildings, COUNT(DISTINCT uprn) AS unique_uprns, COUNT(property_type) AS
```

```

records_with_property_type, COUNT(total_floor_area) AS records_with_floor_area FROM
model.loss_population_1in30;
SELECT COUNT(*) AS records, COUNT(DISTINCT uprn) AS unique_uprns,
MAX(records_per_uprn) AS max_records_per_uprn FROM (SELECT uprn, COUNT(*) AS
records_per_uprn FROM model.loss_population_1in30 GROUP BY uprn) AS x;
SELECT depth_band, COUNT(DISTINCT uprn) AS unique_properties FROM
model.loss_population_1in30 GROUP BY depth_band ORDER BY depth_band;

```

-- How many properties fall into each depth band when we force each property to have one representative flood depth

```

SELECT uprn, COUNT(*) AS flood_records, STRING_AGG(DISTINCT depth_band, ', ' ORDER
BY depth_band) AS depth_bands FROM model.loss_population_1in30 GROUP BY uprn
HAVING COUNT(*) > 1 ORDER BY flood_records DESC;

```

```

SELECT
CASE
WHEN depth_band = '<150mm' THEN 1
WHEN depth_band = '150-300mm' THEN 2
WHEN depth_band = '300-600mm' THEN 3
WHEN depth_band = '600-900mm' THEN 4
WHEN depth_band = '900-1200mm' THEN 5
WHEN depth_band = '1200-2300mm' THEN 6
WHEN depth_band = '>2300mm' THEN 7
END AS depth_rank,
depth_band,
COUNT(*) AS properties
FROM (
SELECT
uprn,
depth_band,
CASE
WHEN depth_band = '<150mm' THEN 1
WHEN depth_band = '150-300mm' THEN 2
WHEN depth_band = '300-600mm' THEN 3
WHEN depth_band = '600-900mm' THEN 4
WHEN depth_band = '900-1200mm' THEN 5
WHEN depth_band = '1200-2300mm' THEN 6
WHEN depth_band = '>2300mm' THEN 7
END AS rank
FROM model.loss_population_1in30
) AS x
WHERE rank = (
SELECT MAX(
CASE
WHEN depth_band = '<150mm' THEN 1

```

```

        WHEN depth_band = '150-300mm' THEN 2
        WHEN depth_band = '300-600mm' THEN 3
        WHEN depth_band = '600-900mm' THEN 4
        WHEN depth_band = '900-1200mm' THEN 5
        WHEN depth_band = '1200-2300mm' THEN 6
        WHEN depth_band = '>2300mm' THEN 7
    END
)
FROM model.loss_population_1in30 y
WHERE y.uprn = x.uprn
)
GROUP BY depth_rank, depth_band
ORDER BY depth_rank;

SELECT
    depth_band,
    COUNT(*) AS properties
FROM (
    SELECT
        uprn,
        depth_band,
        ROW_NUMBER() OVER (
            PARTITION BY uprn
            ORDER BY
                CASE
                    WHEN depth_band = '<150mm' THEN 1
                    WHEN depth_band = '150-300mm' THEN 2
                    WHEN depth_band = '300-600mm' THEN 3
                    WHEN depth_band = '600-900mm' THEN 4
                    WHEN depth_band = '900-1200mm' THEN 5
                    WHEN depth_band = '1200-2300mm' THEN 6
                    WHEN depth_band = '>2300mm' THEN 7
                END DESC
        ) AS rn
    FROM model.loss_population_1in30
) AS ranked
WHERE rn = 1
GROUP BY depth_band
ORDER BY
    CASE
        WHEN depth_band = '<150mm' THEN 1
        WHEN depth_band = '150-300mm' THEN 2
        WHEN depth_band = '300-600mm' THEN 3

```

```

        WHEN depth_band = '600-900mm' THEN 4
        WHEN depth_band = '900-1200mm' THEN 5
        WHEN depth_band = '1200-2300mm' THEN 6
        WHEN depth_band = '>2300mm' THEN 7
    END;

-- Drop old table
DROP TABLE model.loss_population_1in30;

-- Create new table
CREATE TABLE model.loss_population_1in30 AS
SELECT
    building_id,
    uprn,
    flood_id,
    depth_band,
    flood_sour,
    property_type,
    built_form,
    construction_age_band,
    total_floor_area,
    postcode,
    borough_name,
    borough_gss_code
FROM (
    SELECT
        fe.building_id,
        fe.uprn,
        fe.flood_id,
        fe.depth_band,
        fe.flood_sour,
        lbp.property_type,
        lbp.built_form,
        lbp.construction_age_band,
        lbp.total_floor_area,
        lbp.postcode,
        lbp.borough_name,
        lbp.borough_gss_code,
        ROW_NUMBER() OVER (
            PARTITION BY fe.uprn
            ORDER BY
                CASE
                    WHEN fe.depth_band = '<150mm' THEN 1
                    WHEN fe.depth_band = '150-300mm' THEN 2

```

```

        WHEN fe.depth_band = '300-600mm' THEN 3
        WHEN fe.depth_band = '600-900mm' THEN 4
        WHEN fe.depth_band = '900-1200mm' THEN 5
        WHEN fe.depth_band = '1200-2300mm' THEN 6
        WHEN fe.depth_band = '>2300mm' THEN 7
    END DESC
) AS rn
FROM model.flood_exposure_1in30 AS fe
JOIN model.london_building_properties AS lbp
    ON fe.building_id = lbp.building_id
    AND fe.uprn = lbp.uprn
WHERE fe.depth_band <> 'unavailable'
    AND lbp.property_type IS NOT NULL
) AS ranked
WHERE rn = 1;

-- Validate new table
SELECT COUNT(*) AS total_records, COUNT(DISTINCT uprn) AS unique_uprns,
COUNT(DISTINCT building_id) AS unique_buildings, COUNT(*) FILTER (WHERE depth_band
IS NOT NULL) AS records_with_depth, COUNT(*) FILTER (WHERE property_type IS NOT
NULL) AS records_with_property_type, COUNT(*) FILTER (WHERE total_floor_area IS NOT
NULL) AS records_with_floor_area FROM model.loss_population_1in30;

-- Build the same for the higher return period table
CREATE TABLE model.loss_population_1in100_200 AS
SELECT
    building_id,
    uprn,
    flood_id,
    depth_band,
    flood_sour,
    property_type,
    built_form,
    construction_age_band,
    total_floor_area,
    postcode,
    borough_name,
    borough_gss_code
FROM (
    SELECT
        fe.building_id,
        fe.uprn,
        fe.flood_id,
        fe.depth_band,

```

```

fe.flood_sour,
lbp.property_type,
lbp.built_form,
lbp.construction_age_band,
lbp.total_floor_area,
lbp.postcode,
lbp.borough_name,
lbp.borough_gss_code,
ROW_NUMBER() OVER (
    PARTITION BY fe.uprn
    ORDER BY
        CASE
            WHEN fe.depth_band = '<150mm' THEN 1
            WHEN fe.depth_band = '150-300mm' THEN 2
            WHEN fe.depth_band = '300-600mm' THEN 3
            WHEN fe.depth_band = '600-900mm' THEN 4
            WHEN fe.depth_band = '900-1200mm' THEN 5
            WHEN fe.depth_band = '1200-2300mm' THEN 6
            WHEN fe.depth_band = '>2300mm' THEN 7
        END DESC
    ) AS rn
FROM model.flood_exposure_1in100_200 AS fe
JOIN model.london_building_properties AS lbp
    ON fe.building_id = lbp.building_id
    AND fe.uprn = lbp.uprn
WHERE fe.depth_band <> 'unavailable'
    AND lbp.property_type IS NOT NULL
) AS ranked
WHERE rn = 1;

-- Validate table in the same way
SELECT COUNT(*) AS total_records, COUNT(DISTINCT uprn) AS unique_uprns,
COUNT(DISTINCT building_id) AS unique_buildings, COUNT(*) FILTER (WHERE depth_band
IS NOT NULL) AS records_with_depth, COUNT(*) FILTER (WHERE property_type IS NOT
NULL) AS records_with_property_type, COUNT(*) FILTER (WHERE total_floor_area IS NOT
NULL) AS records_with_floor_area FROM model.loss_population_1in100_200;
SELECT depth_band, COUNT(*) AS properties FROM model.loss_population_1in100_200
GROUP BY depth_band ORDER BY CASE WHEN depth_band = '<150mm' THEN 1 WHEN
depth_band = '150-300mm' THEN 2 WHEN depth_band = '300-600mm' THEN 3 WHEN
depth_band = '600-900mm' THEN 4 WHEN depth_band = '900-1200mm' THEN 5 WHEN
depth_band = '1200-2300mm' THEN 6 WHEN depth_band = '>2300mm' THEN 7 END;

-- Create homemade table

```

```

CREATE TABLE model.vulnerability (depth_band TEXT, damage_25yr_2013 NUMERIC,
damage_25yr_2026 NUMERIC, damage_100yr_2013 NUMERIC, damage_100yr_2026
NUMERIC)
-- Populate vulnerability table
INSERT INTO model.vulnerability (
    depth_band,
    damage_25yr_2013,
    damage_25yr_2026,
    damage_100yr_2013,
    damage_100yr_2026
)
VALUES
    ('<150mm', 11975, 17458, 17666, 25755),
    ('150-300mm', 22545, 32868, 27392, 39934),
    ('300-600mm', 29588, 43135, 24133, 35183),
    ('600-900mm', 32922, 47996, 27864, 40622),
    ('900-1200mm', 35511, 51770, 40440, 58956),
    ('1200-2300mm', 36860, 53737, 41614, 60668),
    ('>2300mm', 36860, 53737, 41614, 60668);
-- Validate table
SELECT * FROM model.vulnerability

-- Create the actual loss table for 1-in-30 scenario
CREATE TABLE model.loss_1in30 AS SELECT lp.building_id, lp.uprn, lp.depth_band,
lp.borough_name, lp.borough_gss_code, v.damage_25yr_2026 AS estimated_loss_2026
FROM model.loss_population_1in30 AS lp JOIN model.vulnerability AS v ON lp.depth_band =
v.depth_band;

-- Check table
SELECT COUNT(*) AS total_records, COUNT(DISTINCT uprn) AS unique_uprns,
COUNT(estimated_loss_2026) AS records_with_loss, MIN(estimated_loss_2026) AS
minimum_loss, MAX(estimated_loss_2026) AS maximum_loss FROM model.loss_1in30
SELECT * FROM model.loss_1in30 LIMIT 5;

-- Create the actual loss table for 1-in100/200 scenario
CREATE TABLE model.loss_1in100_200 AS SELECT lp.building_id, lp.uprn, lp.depth_band,
lp.borough_name, lp.borough_gss_code, v.damage_100yr_2026 AS estimated_loss_2026
FROM model.loss_population_1in100_200 AS lp JOIN model.vulnerability AS v ON
lp.depth_band = v.depth_band;

-- Check table
SELECT COUNT(*) AS total_records, COUNT(DISTINCT uprn) AS unique_uprns,
COUNT(estimated_loss_2026) AS records_with_loss, MIN(estimated_loss_2026) AS
minimum_loss, MAX(estimated_loss_2026) AS maximum_loss FROM model.loss_1in100_200;

```

```

SELECT * FROM model.loss_1in100_200 LIMIT 5;

-- END OF DATABASE WORK

-- START OF OUTPUTS

-- Overall exposure and estimated loss- headline summary
SELECT
    '1-in-30 defended' AS scenario,
    COUNT(DISTINCT uprn) AS exposed_properties,
    COUNT(DISTINCT building_id) AS exposed_buildings,
    SUM(estimated_loss_2026) AS total_estimated_loss,
    ROUND(AVG(estimated_loss_2026), 2) AS mean_loss_per_property
FROM model.loss_1in30

UNION ALL

SELECT
    '1-in-100 river / 1-in-200 sea defended' AS scenario,
    COUNT(DISTINCT uprn) AS exposed_properties,
    COUNT(DISTINCT building_id) AS exposed_buildings,
    SUM(estimated_loss_2026) AS total_estimated_loss,
    ROUND(AVG(estimated_loss_2026), 2) AS mean_loss_per_property
FROM model.loss_1in100_200;

-- Damage by depth band (1in30)
SELECT
    depth_band,
    COUNT(*) AS exposed_properties,
    SUM(estimated_loss_2026) AS total_estimated_damage,
    ROUND(AVG(estimated_loss_2026), 2) AS mean_damage_per_property
FROM model.damage_1in30
GROUP BY depth_band
ORDER BY
    CASE depth_band
        WHEN '<150mm' THEN 1
        WHEN '150-300mm' THEN 2
        WHEN '300-600mm' THEN 3
        WHEN '600-900mm' THEN 4
        WHEN '900-1200mm' THEN 5
        WHEN '1200-2300mm' THEN 6
        WHEN '>2300mm' THEN 7
    END;

```

```

--Damage by depth band 1in100/200
SELECT
    depth_band,
    COUNT(*) AS exposed_properties,
    SUM(estimated_loss_2026) AS total_estimated_damage,
    ROUND(AVG(estimated_loss_2026), 2) AS mean_damage_per_property
FROM model.damage_1in100_200
GROUP BY depth_band
ORDER BY
    CASE depth_band
        WHEN '<150mm' THEN 1
        WHEN '150-300mm' THEN 2
        WHEN '300-600mm' THEN 3
        WHEN '600-900mm' THEN 4
        WHEN '900-1200mm' THEN 5
        WHEN '1200-2300mm' THEN 6
        WHEN '>2300mm' THEN 7
    END;

```

```

-- Damage by borough
SELECT
    COALESCE(a.borough_name, b.borough_name) AS borough_name,

    COALESCE(a.exposed_properties, 0) AS properties_1in30,
    COALESCE(a.total_estimated_damage, 0) AS damage_1in30,

    COALESCE(b.exposed_properties, 0) AS properties_1in100_200,
    COALESCE(b.total_estimated_damage, 0) AS damage_1in100_200

FROM (
    SELECT
        borough_name,
        COUNT(*) AS exposed_properties,
        SUM(estimated_loss_2026) AS total_estimated_damage
    FROM model.damage_1in30
    GROUP BY borough_name
) AS a

```

```

FULL OUTER JOIN (
    SELECT
        borough_name,
        COUNT(*) AS exposed_properties,
        SUM(estimated_loss_2026) AS total_estimated_damage
    FROM model.damage_1in100_200

```

```

GROUP BY borough_name
) AS b

ON a.borough_name = b.borough_name

ORDER BY damage_1in100_200 DESC;

--FINAL CHECKS

-- Nulls in the final damage tables
SELECT
'1-in-30' AS scenario,
COUNT(*) AS total_records,
COUNT(*) FILTER (
WHERE building_id IS NULL
OR uprn IS NULL
OR depth_band IS NULL
OR borough_name IS NULL
OR estimated_loss_2026 IS NULL
) AS records_with_nulls
FROM model.damage_1in30

UNION ALL

SELECT
'1-in-100/200' AS scenario,
COUNT(*) AS total_records,
COUNT(*) FILTER (
WHERE building_id IS NULL
OR uprn IS NULL
OR depth_band IS NULL
OR borough_name IS NULL
OR estimated_loss_2026 IS NULL
) AS records_with_nulls
FROM model.damage_1in100_200;

-- Each of the seven depth bands has exactly one vulnerability record
SELECT
depth_band,
COUNT(*) AS vulnerability_records
FROM model.vulnerability
GROUP BY depth_band
ORDER BY
CASE depth_band

```

```

    WHEN '<150mm' THEN 1
    WHEN '150-300mm' THEN 2
    WHEN '300-600mm' THEN 3
    WHEN '600-900mm' THEN 4
    WHEN '900-1200mm' THEN 5
    WHEN '1200-2300mm' THEN 6
    WHEN '>2300mm' THEN 7
END;

```

--MAKE TABLES FOR QGIS MAPS

```

CREATE TABLE model.map_damage_1in30 AS
SELECT
    b.id AS borough_id,
    b.name AS borough_name,
    b.gss_code,
    b.geom,
    COUNT(d.uprn) AS exposed_properties,
    SUM(d.estimated_loss_2026) AS total_estimated_damage
FROM model.boroughs AS b
LEFT JOIN model.damage_1in30 AS d
    ON b.gss_code = d.borough_gss_code
GROUP BY
    b.id,
    b.name,
    b.gss_code,
    b.geom;

```

```

CREATE INDEX map_damage_1in30_geom_idx ON model.map_damage_1in30 USING GIST
(geom);

```

```

CREATE TABLE model.map_damage_1in100_200 AS
SELECT
    b.id AS borough_id,
    b.name AS borough_name,
    b.gss_code,
    b.geom,
    COUNT(d.uprn) AS exposed_properties,
    SUM(d.estimated_loss_2026) AS total_estimated_damage
FROM model.boroughs AS b
LEFT JOIN model.damage_1in100_200 AS d
    ON b.gss_code = d.borough_gss_code
GROUP BY
    b.id,

```

```
b.name,  
b.gss_code,  
b.geom;
```

```
CREATE INDEX map_damage_1in100_200_geom_idx ON model.map_damage_1in100_200  
USING GIST (geom);
```

```
SELECT  
  '1-in-30' AS scenario,  
  COUNT(*) AS boroughs,  
  SUM(exposed_properties) AS exposed_properties,  
  SUM(total_estimated_damage) AS total_damage  
FROM model.map_damage_1in30
```

```
UNION ALL
```

```
SELECT  
  '1-in-100/200' AS scenario,  
  COUNT(*) AS boroughs,  
  SUM(exposed_properties) AS exposed_properties,  
  SUM(total_estimated_damage) AS total_damage  
FROM model.map_damage_1in100_200;
```

```
#FIGURES 1-3 (overview)
```

```
barplot(overview$exposed_properties, names.arg = c("1in30 River/Sea", "1in100 River/1in200 Sea"), xlab = "Flood scenario", ylab = "Number of exposed properties", col = c("green", "blue"), main = "Figure 1 - Number of Exposed Properties under each Flood Scenario")
```

```
barplot(overview$total_estimated_loss, names.arg = c("1in30 River/Sea", "1in100 River/1in200 Sea"), xlab = "Flood scenario", ylab = "Total estimated damage (£)", col = c("green", "blue"), main = "Figure 2 - Total Estimated Damage under each Flood Scenario", yaxt = "n")  
axis(2, at = seq(0, 14000000, by = 2000000), labels = c("£0", "£2m", "£4m", "£6m", "£8m", "£10m", "£12m", "£14m"))
```

```
barplot(overview$mean_loss_per_property, names.arg = c("1in30 River/Sea", "1in100 River/1in200 Sea"), xlab = "Flood scenario", ylab = "Mean estimated damage per property (£)", col = c("green", "blue"), main = "Figure 3 - Mean Estimated Damage under each Flood Scenario", ylim=c(0,40000))
```

```
#FIGURE 4
```

```
# Combine the two scenarios
```

```
damage_combined <- rbind(  
  data.frame(  
    scenario = "1-in-30 River/Sea",  
    estimated_loss_2026 = damage_1in30$estimated_loss_2026  
  ),  
  data.frame(  
    scenario = "1-in-100 River/1-in-200 Sea",  
    estimated_loss_2026 = damage_1in100_200$estimated_loss_2026  
  )  
)
```

```
# Explicitly set scenario order
```

```
damage_combined$scenario <- factor(  
  damage_combined$scenario,  
  levels = c(  
    "1-in-30 River/Sea",  
    "1-in-100 River/1-in-200 Sea"  
  )  
)
```

```
# Create damage bands
```

```
damage_combined$damage_band <- cut(  
  damage_combined$estimated_loss_2026,  
  breaks = c(0, 20000, 30000, 40000, 50000, 60000, Inf),
```

```

labels = c(
  "<£20k",
  "£20k–£30k",
  "£30k–£40k",
  "£40k–£50k",
  "£50k–£60k",
  ">£60k"
),
right = FALSE
)

# Count properties in each damage band
damage_distribution <- table(
  damage_combined$damage_band,
  damage_combined$scenario
)

# Plot distribution
barplot(
  t(damage_distribution),
  beside = TRUE,
  names.arg = c(
    "<£20k",
    "£20k–£30k",
    "£30k–£40k",
    "£40k–£50k",
    "£50k–£60k",
    ">£60k"
  ),
  xlab = "Estimated damage per property",
  ylab = "Number of properties",
  main = "Figure 4 - Distribution of Estimated Damage across Properties",
  col = c("green", "blue"), ylim= c(0,250)
)

legend(
  "topright",
  legend = c(
    "1-in-30 River/Sea",
    "1-in-100 River/1-in-200 Sea"
  ),
  fill = c("green", "blue")
)

```

#FIGURE 5

```
# Convert depth band to ordered numeric position
damage_depth <- rbind(
  data.frame(
    scenario = "1-in-30 River/Sea",
    depth_band = damage_1in30$depth_band,
    estimated_loss_2026 = damage_1in30$estimated_loss_2026
  ),
  data.frame(
    scenario = "1-in-100 River/1-in-200 Sea",
    depth_band = damage_1in100_200$depth_band,
    estimated_loss_2026 = damage_1in100_200$estimated_loss_2026
  )
)

damage_depth$depth_band <- factor(
  damage_depth$depth_band,
  levels = c(
    "<150mm",
    "150-300mm",
    "300-600mm",
    "600-900mm",
    "900-1200mm",
    "1200-2300mm",
    ">2300mm"
  )
)

# Separate the two scenarios
damage_depth_30 <- subset(
  damage_depth,
  scenario == "1-in-30 River/Sea"
)

damage_depth_100 <- subset(
  damage_depth,
  scenario == "1-in-100 River/1-in-200 Sea"
)

# Plot first scenario
plot(
  as.numeric(damage_depth_30$depth_band),
  damage_depth_30$estimated_loss_2026,
```

```

xlim = c(0.5, 7.5),
ylim = c(0, 65000),
xaxt = "n",
yaxt = "n",
xlab = "Flood depth band",
ylab = "Estimated damage per property (£)",
main = "Figure 5 - Flood Depth and Estimated Damage at Property Level",
pch = 16,
col = "green"
)

```

Add second scenario

```

points(
  as.numeric(damage_depth_100$depth_band),
  damage_depth_100$estimated_loss_2026,
  pch = 16,
  col = "blue"
)

```

Custom x-axis

```

axis(
  1,
  at = 1:7,
  labels = c(
    "<150",
    "150-300",
    "300-600",
    "600-900",
    "900-1200",
    "1200-2300",
    ">2300"
  )
)

```

Custom y-axis

```

axis(
  2,
  at = seq(0, 60000, by = 10000),
  labels = c(
    "£0",
    "£10k",
    "£20k",
    "£30k",
    "£40k",
  )
)

```

```

    "£50k",
    "£60k"
  ),
  las = 1
)

legend(
  "topleft",
  legend = c(
    "1-in-30 River/Sea",
    "1-in-100 River/1-in-200 Sea"
  ),
  pch = 16,
  col = c("green", "blue")
)

```

#FIGURE 6

```

# Input scenario data
ep_data <- data.frame(
  return_period = c(1, 30, 100, Inf),
  aep = c(1, 1/30, 1/100, 0),
  loss = c(
    0,
    10942989,
    13736200,
    0
  )
)

# AAL calculation
# Sort by annual exceedance probability
ep_sorted <- ep_data[order(ep_data$aep), ]

# Trapezoidal integration
segment_areas <- diff(ep_sorted$aep) *
(
  head(ep_sorted$loss, -1) +
  tail(ep_sorted$loss, -1)
) / 2

# Illustrative AAL

```

```

aal <- sum(segment_areas)

cat(
  "Illustrative AAL: £",
  format(round(aal, 0), big.mark = ","),
  "\n",
  sep = ""
)

# Increase margins
par(
  mar = c(7, 12, 4, 2) + 0.1,
  xpd = NA
)

plot(
  ep_sorted$aep * 100,
  ep_sorted$loss,
  type = "o",
  pch = 16,
  lwd = 2,
  xlab = "Annual exceedance probability (%)",
  ylab = "",
  main = "Figure 6- Illustrative Loss–Exceedance Probability Curve",
  xaxt = "n",
  yaxt = "n",
  ylim = c(0, 14000000)
)

# Y-axis

y_ticks <- seq(0, 14000000, by = 2000000)

axis(
  2,
  at = y_ticks,
  labels = paste0(
    "£",
    format(y_ticks, big.mark = ",", scientific = FALSE)
  ),
  las = 1
)

mtext(

```

```

    "Estimated total damage (£)",
    side = 2,
    line = 8
)

```

```

# Exact tick positions

```

```

x_ticks <- c(
  0,
  1,
  100 / 30,
  100
)

```

```

axis(
  1,
  at = x_ticks,
  labels = FALSE
)

```

```

# X-axis labels

```

```

text(
  x = 0,
  y = par("usr")[3] - 450000,
  labels = "0%",
  adj = c(0.5, 0.5)
)

```

```

# 1% - slightly higher

```

```

text(
  x = 1.4,
  y = par("usr")[3] - 150000,
  labels = "1%",
  adj = c(0, 0.5)
)

```

```

# 3.33% - lower again and slightly to the right

```

```

text(
  x = 100 / 30,
  y = par("usr")[3] - 450000,
  labels = "3.33%",
  adj = c(0, 0.5)
)

```

```

# 100%
text(
  x = 100,
  y = par("usr")[3] - 450000,
  labels = "100%",
  adj = c(0.5, 0.5)
)

# Scenario labels

text(
  1,
  13736200,
  labels = "1-in-100 river /\n1-in-200 sea",
  pos = 4,
  cex = 0.8
)

text(
  100 / 30,
  10942989,
  labels = "1-in-30 River/Sea",
  pos = 4,
  cex = 0.8
)

# AAL annotation

legend(
  "topright",
  legend = paste0(
    "Illustrative AAL = £",
    format(round(aal, 0), big.mark = ",")
  ),
  bty = "n"
)

```